I/O Multiplexing, Event-Driven Servers, Thread Pools

Lecture 13

Computer Networks and Distributed Programming

Where we are: one flow of control per client

Two versions of the same chat — one process per client, one thread per client — and one cost neither of them avoids.

- Both pay at connection time and keep paying: an address space or a stack, for a client that is silent almost always.
- And we left something open: a `write` to one slow client freezes the whole chat, because the roster lock is held across it.
- Today the flow of control stops being one per client. One watches many connections and serves those that are ready; a fixed set of workers is kept busy by a queue.
- Then the other direction: one datagram, many receivers, with nobody keeping a list.

Waiting on more than one descriptor

- `read` waits on one descriptor. While it waits there, everything else the program can see is invisible to it.
- That is the only reason we had one flow of control per connection: the process, or the thread, existed in order to block.
- `select` turns it around — block once, on a set of descriptors, and return as soon as any of them can be served without blocking.
- What it buys: no thread per socket, no polling loop burning the processor, and one place where the state of every client lives.
- What it does not buy: parallelism. One flow of control still does one thing at a time, on one core.

select

```c
#include <sys/select.h>

int select(int nfds, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,
           struct timeval *timeout);
```

- It blocks until at least one descriptor in the sets can be used, or the timeout expires, or a signal arrives.
- Returns: -1 and `errno` on error, 0 if the timeout expired with nothing ready, otherwise how many descriptors are ready, counted across all three sets.
- `nfds` is not a count: it is the highest descriptor plus one, because the kernel scans the range from 0 to `nfds-1`.

The descriptor sets

An `fd_set` is a bit mask: one bit per descriptor number, and four macros to touch it.

`FD_ZERO(fd_set *set)`	empty the set: always start here
`FD_SET(int fd, fd_set *set)`	put `fd` in the set
`FD_CLR(int fd, fd_set *set)`	take `fd` out of the set
`FD_ISSET(int fd, fd_set *set)`	is `fd` in the set? after the call

- `readfds` for descriptors you want to read from, `writefds` for those you want to write to; `NULL` means do not watch anything of that kind.
- The mask has a fixed size: `FD_SETSIZE` is 1024 on Linux, and a descriptor at or above it cannot be put in a set at all.

When a descriptor is ready

Ready to read — a `read` would return at once:

- bytes have arrived in the receive buffer;
- the peer closed its writing side, so `read` returns 0;
- it is a listening socket and a connection is pending;
- an error is pending on the socket.

Ready to write — a `write` would not block:

- there is room in the send buffer (`SO_SNDLOWAT`, 2048 bytes by default) and the socket is connected;
- a non-blocking `connect` has finished, either way;
- the peer closed its reading side: the `write` returns with `EPIPE`;
- an error is pending on the socket.

Ready means will not block, not will succeed: an error is a perfectly good reason to be ready. `exceptfds` is almost always `NULL` — on Linux the only thing it reports is out-of-band data.

The timeout

```c
struct timeval {
    time_t      tv_sec;      /* seconds */
    suseconds_t tv_usec;     /* microseconds, not milliseconds */
};

struct timeval tv   = { .tv_sec = 2, .tv_usec = 500000 };   /* 2.5 seconds */
struct timeval zero = { .tv_sec = 0, .tv_usec = 0 };

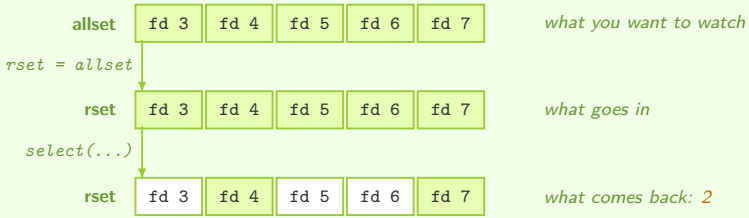
select(nfds, &rset, NULL, NULL, NULL);    /* wait for ever */
select(nfds, &rset, NULL, NULL, &zero);   /* poll: return immediately */
select(nfds, &rset, NULL, NULL, &tv);     /* wait at most 2.5 s */
```

On Linux `select` writes back into that structure the time it did not use: inside a loop it must be filled in again before every call, or the wait shrinks to nothing.

select consumes the sets

You build the set once, before the loop, and call `select` in the loop. What does the second call wait for?

Only for the descriptors that were ready the first time. `select` overwrites the set with the subset that is ready, and clears everything else — the sets are arguments and results.



The three descriptors that were not ready have been cleared. A set built once, outside the loop, waits the second time only for those that were ready the first time — and after that, for nothing at all.

The client that cannot hear the server

Your client reads a line, sends it, and reads the answer. The server also speaks on its own — one line every ten seconds. When does the user see it?

When he types something. The client is sitting in `read` on the keyboard, and the lines from the server pile up in the socket buffer until it comes back.

- It is not a bug in the server: the client can only be blocked in one of the two places, and it chose the wrong one.
- A second thread would fix it — one per descriptor, again, with a shared screen to write on.
- For two descriptors it is one call instead.

Watching the keyboard and the socket

```c
static void chat_client(int fd) {
    char buf[256];
    fd_set rset;
    int nfds = (fd > STDIN_FILENO ? fd : STDIN_FILENO) + 1;

    for (;;) {
        FD_ZERO(&rset);                          /* rebuilt at every turn */
        FD_SET(STDIN_FILENO, &rset);
        FD_SET(fd, &rset);

        if (select(nfds, &rset, NULL, NULL, NULL) < 0) {
            if (errno == EINTR) continue;      /* a signal, not an error */
            perror("select"); return;
        }
```

Two descriptors, one wait. The client is now blocked in both places at once, which is the whole point.

Which of the two was ready

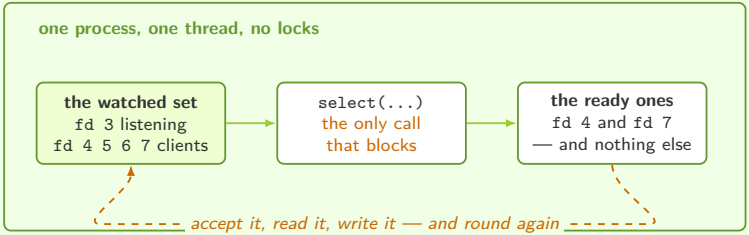
```
        if (FD_ISSET(STDIN_FILENO, &rset)) {          /* the user typed */
            ssize_t n = read(STDIN_FILENO, buf, sizeof buf);
            if (n <= 0) return;                       /* end of input: we are done */
            if (write(fd, buf, n) != n) { perror("write"); return; }
        }

        if (FD_ISSET(fd, &rset)) {                    /* the server spoke */
            ssize_t n = read(fd, buf, sizeof buf);
            if (n == 0) { fprintf(stderr, "server closed\n"); return; }
            if (n < 0)  { perror("read"); return; }
            write(STDOUT_FILENO, buf, n);
        }
    }
}
```

Two `if`, not an `if/else`: `select` may report both, and the second one would be lost until the next line is typed.

One flow of control, many connections

The same server, with the loop turned inside out: it no longer waits for a client, it waits for whichever client speaks first.



Every client is a slot in an array and a bit in a set. There is nothing to lock, because nothing else is running — and nothing on the return path may wait, because everybody waits with it.

The server's state

```
#define MAXC 64
static int clients[MAXC];                   /* connected sockets, -1 = free slot */

int main(void) {
    int listen_fd = listening_socket(5000);
    fd_set allset, rset;                    /* the master set, and the copy */
    int maxfd = listen_fd;

    for (int i = 0; i < MAXC; i++) clients[i] = -1;
    FD_ZERO(&allset);
    FD_SET(listen_fd, &allset);             /* the door is a descriptor like the others */

    for (;;) {
        rset = allset;                       /* struct assignment: a copy */
        int ready = select(maxfd + 1, &rset, NULL, NULL, NULL);
        if (ready < 0) { if (errno == EINTR) continue; perror("select"); break; }
```

Same array of clients as the threaded version, without the mutex: nothing else is running.

A connection arrives on the listening socket

```
        if (FD_ISSET(listen_fd, &rset)) {
            int fd = accept(listen_fd, NULL, NULL);
            if (fd < 0) { perror("accept"); continue; }

            int slot = -1;
            for (int i = 0; i < MAXC; i++)
                if (clients[i] == -1) { clients[i] = fd; slot = i; break; }

            if (slot < 0) { write(fd, "chat is full\n", 13); close(fd); }
            else {
                FD_SET(fd, &allset);              /* watched from the next turn on */
                if (fd > maxfd) maxfd = fd;
            }
            if (--ready == 0) continue;            /* nothing else was ready */
        }
```

A readable listening socket means a pending connection, so this `accept` returns at once.

Serving the connections that are ready

```
1        for (int i = 0; i < MAXC && ready > 0; i++) {
2            int fd = clients[i];
3            if (fd == -1 || !FD_ISSET(fd, &rset)) continue;
4            --ready;
5
6            char buf[256];
7            ssize_t n = read(fd, buf, sizeof buf);
8            if (n > 0) {
9                broadcast(i, buf, n);              /* everybody except i */
10           } else {                               /* 0: it left. <0: it broke */
11               FD_CLR(fd, &allset);
12               close(fd);
13               clients[i] = -1;
14           }
15       }
```

`broadcast` is the `for` loop of the threaded version minus the mutex. `maxfd` is not lowered when a client leaves: the cost is a slightly longer scan, and the alternative is a search on every departure.

Nothing in this loop may block

A client sends half a line, one byte at a time, one byte per second. Your `read` returned six bytes and the line is not finished. What now?

You take the six bytes, put them in that client's own buffer and go back to `select`. Waiting for the rest is waiting for one client with everybody else's time.

- So each client needs its own input buffer: the loop keeps the state that a thread used to keep on its stack.
- And ready is not a promise: between the `select` and the `accept` the client can send a reset, and the `accept` blocks — with the whole server inside it.
- Hence `fcntl(fd, F_SETFL, O_NONBLOCK)`, or `MSG_DONTWAIT` on the single call: EAGAIN instead of a wait, and the loop goes on.

The slow client, answered

```
static struct { char buf[4096]; size_t len; } out[MAXC];  /* pending bytes, per client */
static fd_set wall;                               /* who still has something to send */

static void queue(int i, const char *msg, size_t n) {     /* this never blocks */
    if (out[i].len + n > sizeof out[i].buf) { drop(i); return; }  /* too slow: drop it */
    memcpy(out[i].buf + out[i].len, msg, n);
    out[i].len += n;
    FD_SET(clients[i], &wall);                    /* tell me when it can take bytes */
}

/* and in the loop, for each i that select reported writable */
ssize_t w = write(clients[i], out[i].buf, out[i].len);
if (w > 0) { out[i].len -= w; memmove(out[i].buf, out[i].buf + w, out[i].len); }
if (out[i].len == 0) FD_CLR(clients[i], &wall);   /* nothing left: stop asking */
```

`select` now gets a write set too. The slow client fills its own buffer and is dropped when it overflows: nobody else waits for it.

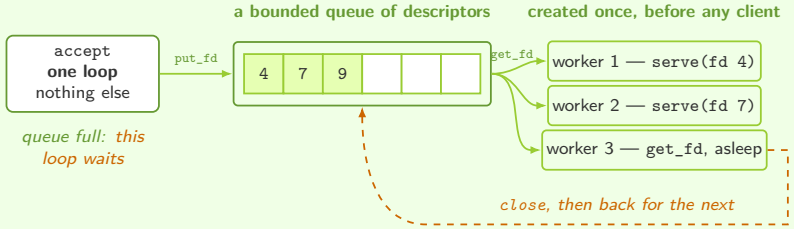
What select costs

- The sets are copied into the kernel at every call, and the kernel walks the whole range from 0 to `nfds-1`: the cost follows the highest descriptor, not the number of ready ones.
- When it returns you walk your own array to find who is ready — a second linear pass over clients that mostly said nothing.
- And `FD_SETSIZE` is 1024: a server with ten thousand connections cannot even name its descriptors in a set.
- `poll` takes an array instead of a bit mask — no fixed limit, still linear. `epoll` on Linux and `kqueue` on BSD register the descriptors once and return only what is ready.

The shape of the loop does not change: what changes is the call in the middle. Everything on these frames still applies.

A fixed set of workers

The event loop removed the flow of control per client and left one for everybody. A pool keeps several, but a fixed number, created before the first client arrives.



A connection costs one enqueue and one wakeup: nothing is created, and nothing is destroyed, when a client arrives.

The queue is the bounded buffer, holding descriptors

```c
#define QSIZE 128

typedef struct {
    int fd[QSIZE];                          /* descriptors instead of integers */
    int in, out, count;
    pthread_mutex_t mtx;
    pthread_cond_t  not_full, not_empty;    /* the same two waits as before */
} queue_t;

void put_fd(queue_t *q, int fd);            /* blocks while the queue is full */
int  get_fd(queue_t *q);                    /* blocks while it is empty */
```

It is the producer-consumer buffer, unchanged: one mutex over the state, one condition variable for each thing somebody waits for. What travels through it is now a connection.

The accept loop is the producer

```c
int main(void) {
    int listen_fd = listening_socket(5000);
    queue_t q; queue_init(&q);
    pthread_t tid;

    for (int i = 0; i < NWORKERS; i++)          /* once, before any client */
        if (pthread_create(&tid, NULL, worker, &q) == 0) pthread_detach(tid);

    for (;;) {
        int fd = accept(listen_fd, NULL, NULL);
        if (fd < 0) { if (errno == EINTR) continue; perror("accept"); continue; }
        put_fd(&q, fd);                         /* the only thing the loop does */
    }
}
```

No `fork`, no `pthread_create` in the loop, no `malloc` for an argument: a connection costs one enqueue.

The worker is the consumer

```c
static void *worker(void *arg) {
    queue_t *q = arg;

    for (;;) {
        int fd = get_fd(q);                 /* asleep here while there is nothing */
        if (fd == STOP) break;                 /* the kill pill: one per worker */

        serve(fd);                             /* the sequential server's function */
        close(fd);
    }
    return NULL;
}
```

The thread was there before the client and is there after it: nothing is created or destroyed per connection. And `serve()` has now been through three different loops without being touched.

How many workers, and what happens when the queue fills

- The number is not «as many as the clients»: it is how many can work at once — around the number of cores for work that computes, more when the workers spend their time blocked in `read`.
- The queue is bounded on purpose. When it fills, `put_fd` blocks the accept loop; connections then pile up in the `listen` queue, and when that fills too the kernel refuses them.
- That chain is back pressure: a server that cannot keep up says so, at the door, instead of allocating memory until it dies.
- An unbounded queue only moves the failure further in — it accepts everybody and answers nobody, and the client has given up long before its turn comes.

So the number to measure is not throughput but how long a request waits before a worker picks it up.

All sixteen workers are talking to somebody

Sixteen workers, sixteen chat clients connected and silent. The seventeenth connects. When is it served?

Never. Every worker is inside `serve()`, blocked in `read` on its own client, and `serve()` returns only when that client leaves. The queue is not full — it is simply never read.

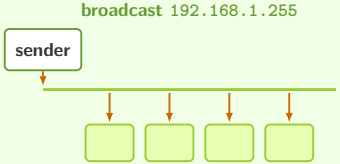
- A pool whose job is a connection serves as many clients as it has workers. It is the right shape when the job ends: an HTTP request, a query, a file to send.
- For connections that last, either you keep one flow per client and bound their number — the counting semaphore we already used — or the job becomes the message.
- Which needs somebody to notice that a message arrived. That is the loop of the previous frames: one that watches, a pool that works.

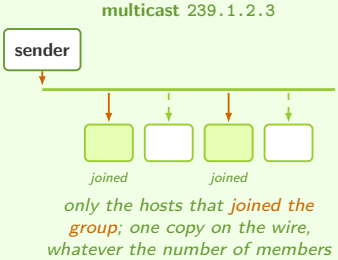
Three shapes of a server

	one per client	**one loop**	**fixed pool**
cost of a new client	a stack	a slot and a bit	one enqueue
how many at once	thousands	tens of thousands	as many as the workers
a blocking call	stops that client	stops everybody	occupies one worker
shared state	needs a mutex	needs nothing	needs a mutex
uses several cores	yes	no	yes

They are not alternatives for long: a real server runs one loop per core, each with a pool behind it for the work that must not stop the loop.

One sender, many receivers





Both are datagrams: there is no connection with many peers, so no TCP. And IPv4 only — IPv6 removed broadcast, and gives its jobs to multicast.

Broadcast: the two ends

```c
/* the sender */
int fd = socket(AF_INET, SOCK_DGRAM, 0);
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_BROADCAST, &yes, sizeof yes);  /* off by default */
to.sin_port        = htons(5000);
inet_pton(AF_INET, "192.168.1.255", &to.sin_addr);   /* this subnet's broadcast */
sendto(fd, msg, len, 0, (struct sockaddr *) &to, sizeof to);

/* the receiver: an ordinary UDP server */
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
me.sin_addr.s_addr = htonl(INADDR_ANY);              /* not the broadcast address */
me.sin_port        = htons(5000);                    /* the same port as the sender */
bind(fd, (struct sockaddr *) &me, sizeof me);
recvfrom(fd, buf, sizeof buf, 0, (struct sockaddr *) &from, &flen);
```

`SO_BROADCAST` exists so that nobody floods a LAN by mistyping an address: you have to say that you meant it.

The broadcast that does not arrive

Your program runs, `sendto` returns the number of bytes it sent, and nobody receives anything. Where did it go?

Out of the socket, and no further. `sendto` reports what left your machine: on UDP that is all it can ever promise.

- `255.255.255.255` is the limited broadcast: it never leaves the interface it was sent on. Routers do not forward broadcast at all — it stops at the subnet, by design.
- The subnet broadcast has to be that subnet's: read it, do not guess it.
- A virtual machine in NAT mode is on a private network of its own, and that NAT does not forward broadcast either. In bridged mode it gets an address on the real LAN and the room receives it — if the Wi-Fi driver allows bridging.

```
$ ip addr show ens33 | grep inet     brd is the address you want
```

Multicast addresses

Definition

A multicast group is an IPv4 address between 224.0.0.0 and 239.255.255.255. It names a conversation, not a host: nobody owns it, and any host may join it.

`224.0.0.0/24`	link-local: never forwarded — routing protocols live here
`224.0.1.0–238....`	global scope: needs routers configured for it
`239.0.0.0/8`	administratively scoped: local to one organisation

Use `239.x.y.z` for anything of your own. The other limit on how far a datagram travels is its TTL, which for multicast the sender chooses explicitly.

Joining a group

```
int fd = socket(AF_INET, SOCK_DGRAM, 0);
int yes = 1;
setsockopt(fd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof yes);
setsockopt(fd, SOL_SOCKET, SO_REUSEPORT, &yes, sizeof yes);

me.sin_addr.s_addr = htonl(INADDR_ANY);
me.sin_port        = htons(5000);
bind(fd, (struct sockaddr *) &me, sizeof me);       /* the port, not the group */

struct ip_mreq mreq;
inet_pton(AF_INET, "239.1.2.3", &mreq.imr_multiaddr); /* which group */
mreq.imr_interface.s_addr = htonl(INADDR_ANY);      /* on which interface */
setsockopt(fd, IPPROTO_IP, IP_ADD_MEMBERSHIP, &mreq, sizeof mreq);
```

`bind` chooses the port, the join chooses the group: without it the kernel throws the datagrams away even when they arrive on the wire. `SO_REUSEADDR` reuses a port left behind by an earlier socket; `SO_REUSEPORT` lets several live sockets hold the same port at the same time — which is what running two peers on one machine needs.

Sending to a group

```c
int fd = socket(AF_INET, SOCK_DGRAM, 0);       /* an ordinary UDP socket */

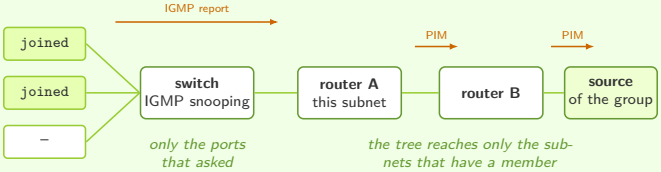
unsigned char ttl = 1;                         /* 1: this subnet and no further */
setsockopt(fd, IPPROTO_IP, IP_MULTICAST_TTL, &ttl, sizeof ttl);

unsigned char loop = 1;                        /* also to members on this host */
setsockopt(fd, IPPROTO_IP, IP_MULTICAST_LOOP, &loop, sizeof loop);

to.sin_family      = AF_INET;
to.sin_port        = htons(5000);
inet_pton(AF_INET, "239.1.2.3", &to.sin_addr);
sendto(fd, msg, len, 0, (struct sockaddr *) &to, sizeof to);
```

The sender does not join anything, connects to nothing and does not know who is listening: this code is identical whether the group has zero members or two hundred.

Who makes multicast selective



- IGMP: the join is not only local: the kernel announces the group to the router on the subnet, and repeats when asked.
- PIM: routers build a tree between subnets, so that a subnet with no member receives nothing.
- Snooping: the switch forwards the group only to the ports that asked; without it, it floods the LAN and each kernel does the filtering.
- None of the three is access control: any host on the path may join and read everything. If it must be private, encrypt the payload.

Summary

- `select` blocks on a set instead of a descriptor, and returns those that can be served without waiting: one flow of control for every connection there is.
- The sets are arguments and results, so the master set is copied at every turn — and nothing on the path from `select` back to `select` may block.
- That is what finally answers the slow client: an output queue per connection, drained when the socket says it is writable.
- A pool pays for its threads once and keeps them: the accept loop produces, the workers consume, and a bounded queue turns overload into refusal instead of exhaustion.
- A pool serves as many long-lived connections as it has workers — so the job is a message, and something else notices that a message arrived.
- Broadcast reaches every host on the subnet and stops there; multicast reaches those that joined a group, over UDP, with one copy on the wire and no idea of who is listening.

Try it

Two programs

One: run `es_fork_pipe_select.c`, where parent and child both watch the keyboard and the parent forwards what it reads through a pipe. Type a few strings and explain what you see. Two: write a multicast peer — one UDP socket, joined to `239.1.2.3:5000`, with `select` on `stdin` and on the socket. What you type goes to the group, what arrives goes to the screen. No server: every peer is both.

```
$ ./peer 239.1.2.3 5000      three of them on one machine, then two machines
```

In the first program, who gets the line you typed — and can you tell in advance? In the second, why do you see your own messages, and what are the two ways to stop seeing them?

Iteration 4: an event-driven chat

Where you restart from

Your threaded chat: it works, and one client that stops reading freezes everybody, because the broadcast blocks with the roster lock held.

- What changes: one thread. No `pthread_create`, no mutex: the roster is a plain array, and `accept` happens when the listening socket is readable.
- What is new: nothing may block — the descriptors go non-blocking, and each client gets an output queue that enters the write set only when it has something in it.
- Then measure: memory with 200 connected clients, against the threaded version.

```
$ ss -tn | grep :5000 | wc -l     then suspend one client with ^Z
```

What happens to the other 199 now? And what does your server's memory do while that client's queue grows?

Iteration 5: a pool behind the loop

Where you restart from

Iteration 4: one flow of control for everybody. A client that makes the server work — a search, a file read from disk — stops every other conversation for as long as it takes.

- What changes: the loop stops serving. When a descriptor is readable it leaves the watched set and goes into the queue; a worker reads it, does the work, queues the answer and hands the descriptor back.
- Why not a pool of connections: because a chat connection never ends: sixteen workers would serve sixteen clients for ever. The job is a message.
- Then measure: with eight workers and one client asking for something slow, the delay seen by the others.

Why must the descriptor leave the watched set while a worker holds it? And after how many workers does the answer stop improving?